

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work n. 3
of Operating Systems

The Command Line: Shell, Pipes and Text Filters

Supervisers:

Dumitrașcu Marius, *University assistant*

Reițman Patricia, *University assistant*

DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Introduction **3**

2 Before we start **4**

3 Tasks **4**

3.1 Move fast, find help 4

3.1.1 Observations 7

3.2 What the shell does to your line 7

3.2.1 Observations 8

3.3 Redirection, chaining and jobs 9

3.3.1 Observations 10

3.4 Answer questions with one-liners 10

3.5 My first script 11

4 CONCLUSIONS **14**

1 Introduction

The command line provides direct access to the operating system through a shell. Bash can interpret commands, expand filenames and variables, connect programs through pipes, redirect input and output, and automate repeated operations. These facilities are particularly useful when working with files, processes, and structured text.

The work focuses on navigating Bash efficiently, examining how it interprets command lines, and combining standard Unix utilities to answer questions about a web access log. It also covers writing a small shell script and scheduling its execution.

2 Before we start

The commands are executed in Bash on an Ubuntu virtual machine. A working directory is created at `~/os-lab3`, together with an artificial web access log used for the text-processing exercises.

Each log entry contains a date, time, username, request status, and page. A Bash loop generates 500 entries; the status alternates according to the loop counter, while usernames and pages repeat in fixed cycles. The first three lines and the line count are displayed to check the generated data.



Figure 2.1 - Generating the access log and verifying its 500 entries

The resulting `access.log` contains 500 requests and is ready for further analysis.

3 Tasks

3.1 Move fast, find help

Bash provides filename and directory completion using the Tab key. Entering an incomplete path under `/usr/sh` and pressing Tab completes it to `/usr/share/`. Likewise, entering `cd /et` and pressing Tab completes the path to `/etc/`. Completion reduces repetitive typing and helps locate existing paths.

The command history can be searched without repeatedly using the arrow keys. After running `echo`, `ls`, and `wc`, pressing `Ctrl+R` and entering `ec` locates the earlier `echo something` command. Pressing Enter executes that command again.



Figure 3.1 - Command history and reverse incremental search

Commands may be implemented inside Bash or provided by separate executables. The `type` command identifies `cd` and `echo` as shell built-ins. In the current environment, `ls` is an alias for `ls --color=auto`, and the executable is located at `/usr/bin/ls`. The shell searches directories named in its `PATH` variable to find external commands. The current `PATH` contains ten entries, including a repeated `/snap/bin` entry.

The `tr` utility is used to replace the colon separators in `PATH` with newline characters, making each directory visible on its own line. An attempt to run `nosuchprog` fails with exit status 127. The variable `$?` holds the exit status of the most recently completed foreground command.

```
misa@UbuntuVM:~/os-lab3$ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
misa@UbuntuVM:~/os-lab3$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
misa@UbuntuVM:~/os-lab3$ echo $PATH | tr ':' '\n'
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
misa@UbuntuVM:~/os-lab3$ nosuchprog ; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
misa@UbuntuVM:~/os-lab3$
```

Figure 3.2 - Built-in commands, executable lookup, PATH and exit status

Manual pages are available directly in the terminal. In `man ls`, searching for `sort` reveals the `-t` option and the `--sort=time` form, which order entries by modification time. The `n` key moves to the next match, and `q` closes the manual.

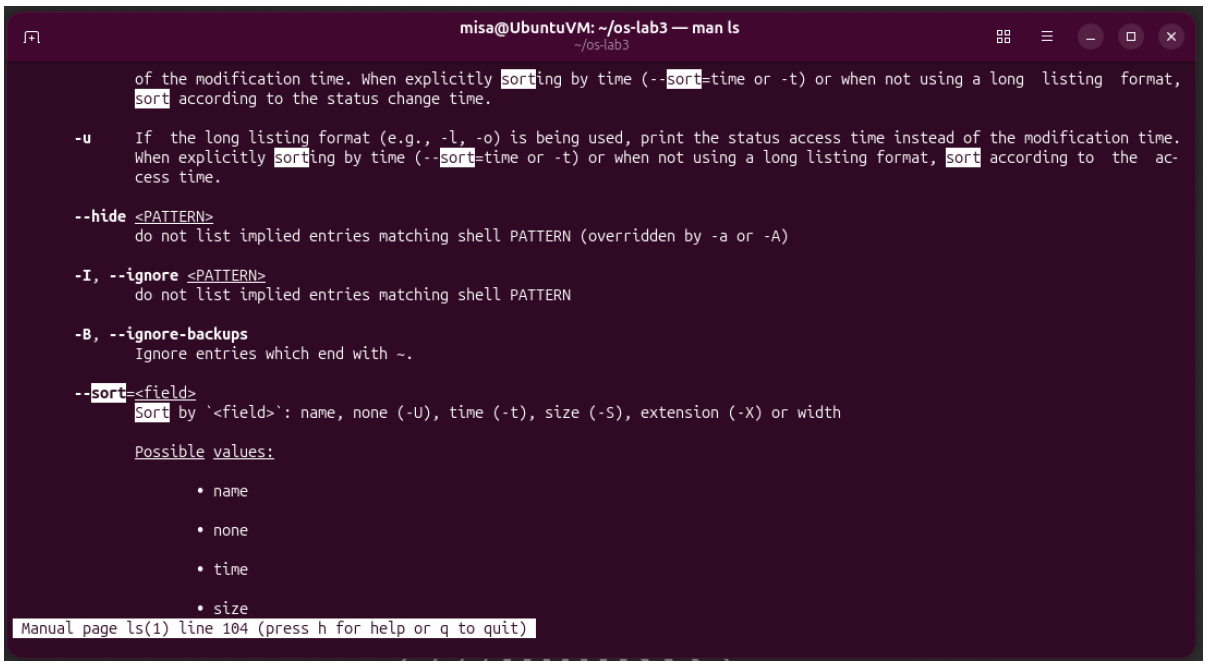


Figure 3.3 - Finding the sorting option in the ls manual page

Additional help is available through `ls --help`, `man -k`, and the shell's `help` command. The first displays option documentation, `man -k` searches manual descriptions by keyword, and `help cd` describes Bash's built-in directory command.

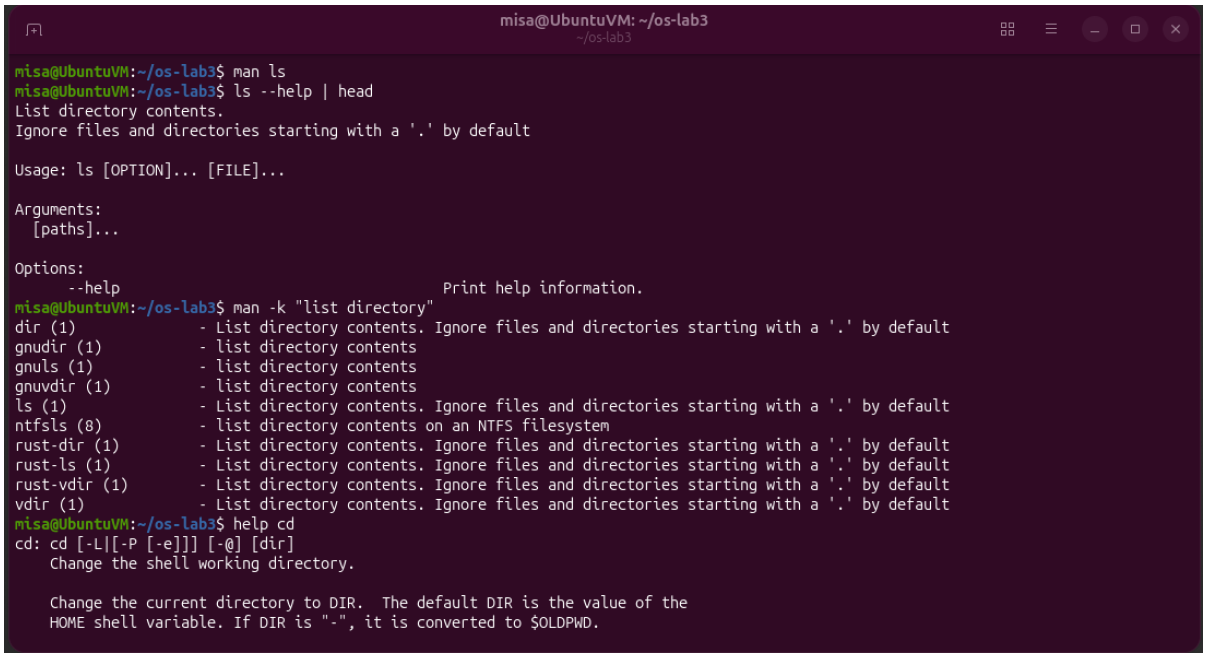


Figure 3.4 - Command help, manual keyword search and built-in help

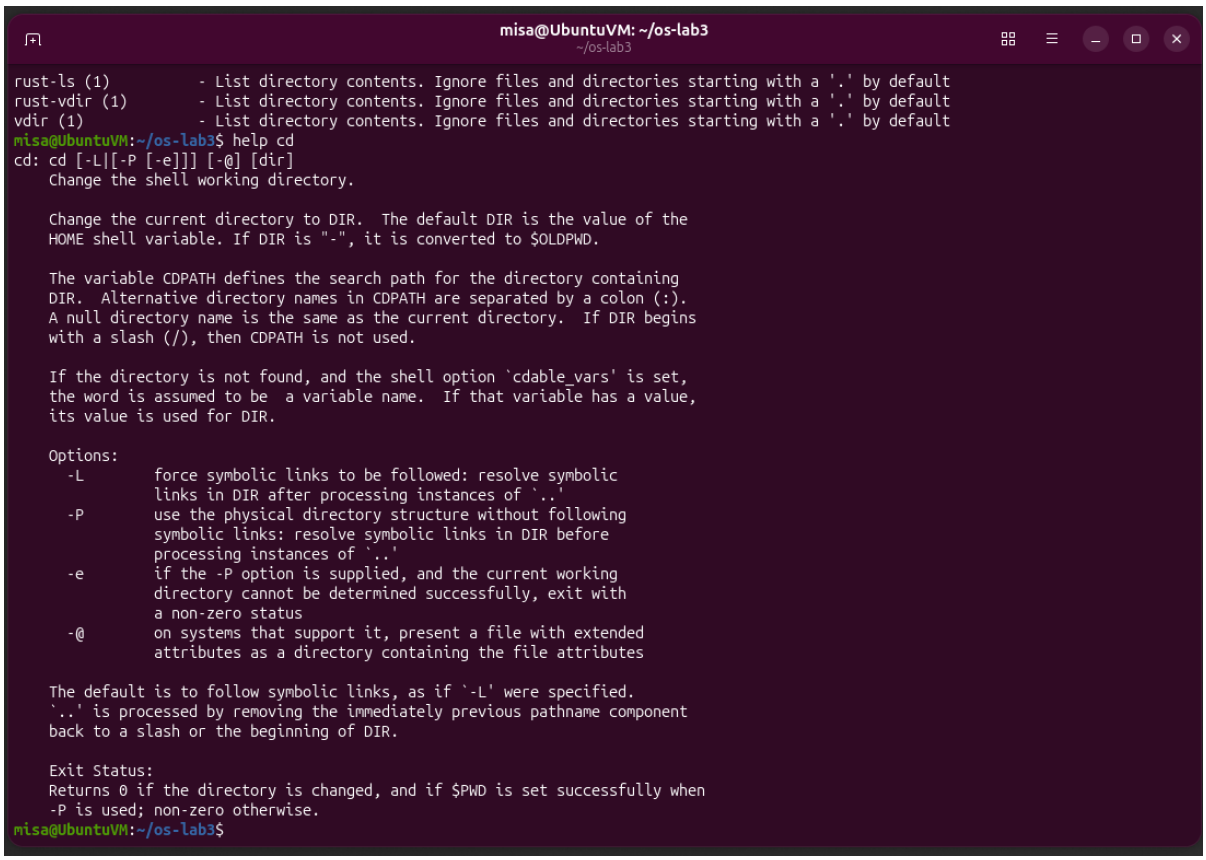


Figure 3.5 - Additional documentation for the cd shell built-in

3.1.1 Observations

1. **Which of cd, ls and echo are built-ins, and why must cd be one?** `cd` and `echo` are Bash built-ins; `ls` resolves to an alias that invokes an external executable. Changing a directory must affect the current shell process. A separate program would change only its own working directory.
2. **How many directories are in PATH, and where is ls?** There are ten `PATH` entries, with nine distinct directories. The `ls` executable is at `/usr/bin/ls`.
3. **What exit code does a missing command return?** Bash returned 127 for `nosuchprog`.
4. **Which ls option sorts by modification time?** The option is `-t`, also available as `--sort=time`.

3.2 What the shell does to your line

A separate `play` directory is populated with C source filenames, text filenames, and a filename containing a space. The shell processes expansions and quoting before it invokes a command. This is visible even with `echo`, which simply prints the arguments it receives.

The patterns `*.c`, `file?.txt`, and `[a-b]*` are expanded by Bash to matching filenames. The asterisk matches any sequence of characters; a question mark matches one character; and a bracket expression matches one character from a specified set or range. When no filename matches `*.xyz`, Bash leaves that pattern unchanged under its default settings.

A filename containing spaces must be passed as one argument. The unquoted command `ls alfa`

`beta.txt` searches for two separate names and fails. Quoting the filename makes the command work. Bash also distinguishes single quotes, double quotes, and backslash escaping: single quotes preserve text, double quotes permit variable expansion, and a backslash protects the next character from special interpretation.

Brace expansion generates the numbers 1 through 5 and the filenames `log1.txt` to `log3.txt`. Arithmetic expansion evaluates `7 * 6` to 42. Command substitution inserts the output of `date` and `whoami` into a larger string; the date format `%A` requests the full weekday name.

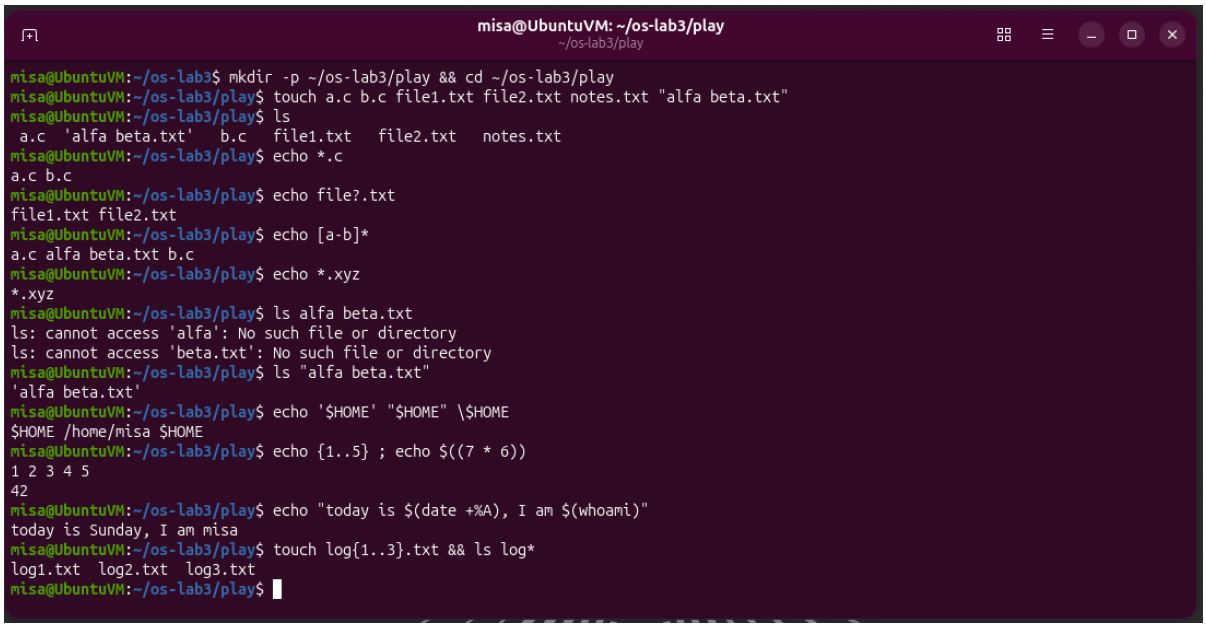


Figure 3.6 - Filename patterns, quoting and Bash expansions

3.2.1 Observations

The predicted and observed outputs agreed for most commands. Three details required correction: an unmatched `*.xyz` pattern was printed literally, the backslash preceding `$HOME` disappeared while preventing expansion, and `date +%A` produced the full weekday name.

1. `echo *.c`: predicted and observed `a.c b.c`.
2. `echo file?.txt`: predicted and observed `file1.txt file2.txt`.
3. `echo [a-b]*`: predicted and observed the names beginning with `a` or `b`, including `alfa beta.txt`.
4. `echo *.xyz`: predicted an empty result; observed the literal pattern `*.xyz`, because no file matched it.
5. `ls alfa beta.txt`: predicted and observed two `No such file or directory` errors.
6. `ls "alfa beta.txt"`: predicted and observed the single matching filename. The displayed quotes are formatting added by `ls`.
7. `echo` with three forms of `$HOME`: predicted literal text for single quotes and a path for double quotes; observed `$HOME /home/misa $HOME`. The backslash is consumed during escaping rather than printed.
8. Brace and arithmetic expansion: predicted and observed `1 2 3 4 5` followed by 42.

9. Command substitution: expected an inserted day and username, without knowing the weekday format; observed `today is Sunday, I am misa`.
10. The `touch` command with brace expansion: predicted and observed three files named `log1.txt`, `log2.txt`, and `log3.txt`.

Bash performs glob expansion before executing `echo`. The unquoted filename with a space is split into two arguments; it can be fixed using `ls "alfa beta.txt"` or by escaping the space. Single quotes prevent expansion; double quotes allow variable and command substitution while preserving spaces inside the argument.

3.3 Redirection, chaining and jobs

Unix processes receive three standard file descriptors: standard input (0), standard output (1), and standard error (2). Redirection changes where these streams are read or written. Redirecting output with > creates or overwrites a file, while >> appends to it.

The command that lists `/etc` and a nonexistent `/nope` path writes the directory listing to `out.txt` and the error message to `err.txt` using 2>. The listing contains 238 lines in the recorded run. Redirecting both streams with `> all.txt 2>&1` places them in the same file: `2>&1` makes descriptor 2 point to the current destination of descriptor 1.

Redirections are applied from left to right. In the reversed form `2>&1 > reverse.txt`, standard error continues to point to the terminal while standard output goes to the file. The two forms can therefore produce different output locations even though they contain the same operators.



Figure 3.7 - Comparing the two redirection orders

Input redirection with < supplies file contents to standard input. Thus, `wc -l < access.log` prints `500` without a filename, whereas naming the file as an argument also displays `access.log`. The special file

`/dev/null` can discard unwanted error output.

The shell also supports conditional execution. A semicolon separates commands without imposing a success condition. The `&&` operator runs the next command only if the previous command succeeded; `||` runs it only if the previous command failed. The variable `$?` provides the last exit status, which is 2 in the recorded missing-path example.

Finally, appending `&` starts `sleep 100` as a background job. The `jobs` command lists active shell jobs, and `kill %1` sends a termination signal to job 1.

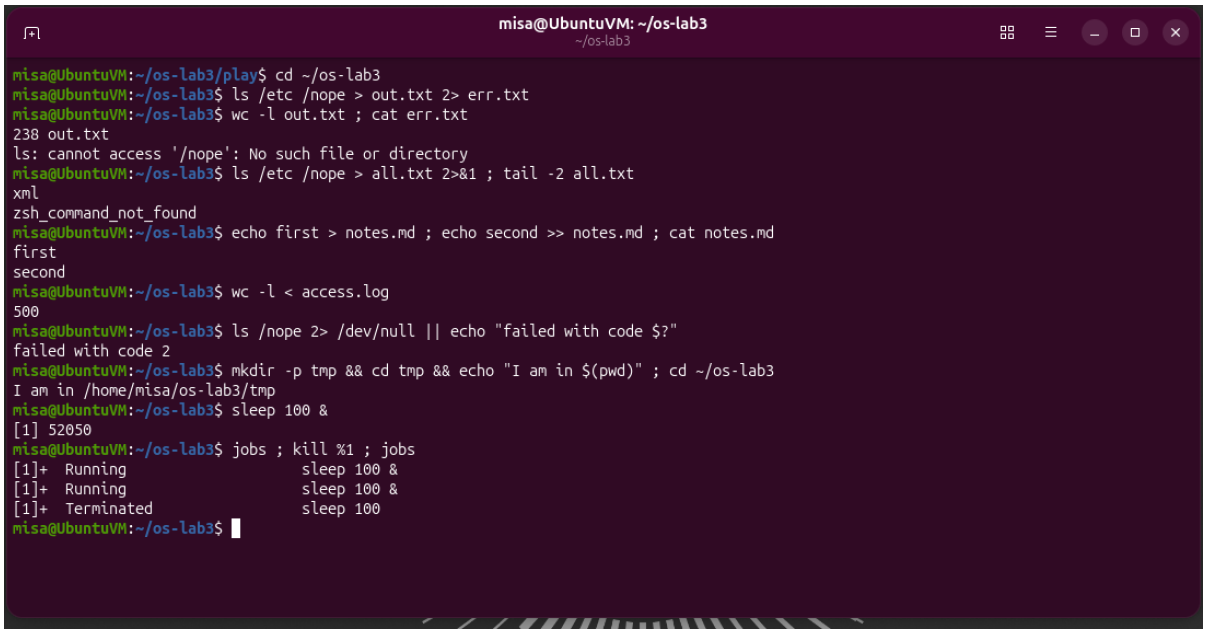


Figure 3.8 - Output redirection, conditional commands and background jobs

3.3.1 Observations

1. **Which file received the error and which the listing?** The directory listing was written to `out.txt`; the error about `/nope` was written to `err.txt`.
2. **How do the two orders of redirection differ?** With `> all.txt 2>&1`, both streams go to the file. With `2>&1 > reverse.txt`, the error remains on the terminal and the directory listing goes to `reverse.txt`; Bash processes the redirections left to right.
3. **Why does input redirection change the wc output?** With `< access.log`, `wc` reads unnamed standard input. With `wc -l access.log`, it knows and prints the filename.
4. **What do the chaining operators mean?** `;` runs commands in sequence; `&&` requires success before continuing; `||` requires failure before continuing.

3.4 Answer questions with one-liners

The access log is analysed through pipelines of small text-processing utilities. `grep` selects matching records, `cut` extracts fields, `sort` orders values, `uniq -c` counts adjacent equal values, and `wc -l` counts

lines. Every question is represented by one executable command line in `oneliners.sh`.

The 500 requests contain 100 failures. Four distinct pages occur: `/page0`, `/page1`, `/page2`, and `/page3`, each requested 125 times. The highest number of failed requests is shared by `user3` and `user5`, with 15 each. In total, `user3` made 72 requests, of which 15 failed. The last three failed requests are `10:10 user0`, `10:15 user5`, and `10:20 user3`.

The remaining two questions concern the local operating system. The seventh field of `/etc/passwd` records the configured account shell; counting its distinct values reveals how many accounts use each shell on the current machine. The results depend on the machine's installed accounts. Finally, `ls -l /etc | wc -l` counts one additional line because the long listing begins with a `total` header.

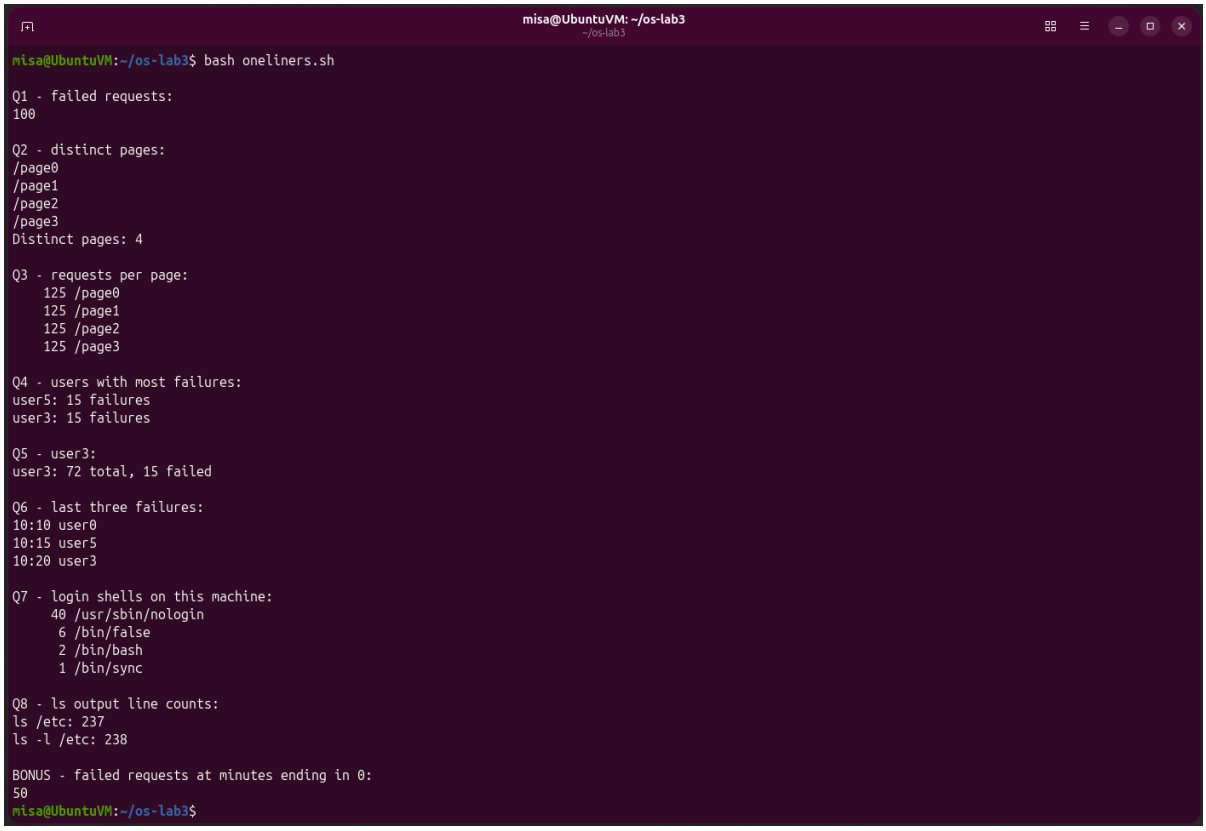


Figure 3.9 - Results of the oneliners

An additional extended regular expression selects failed requests whose minute ends in zero. The filter matches the six minute values 00, 10, 20, 30, 40, and 50, and produces 50 matching records in the generated log.

3.5 My first script

The most useful filters are collected in `report.sh`. The script accepts the path of a log file as its first argument, checks whether the file exists, and prints the total number of requests and failures. A loop then counts failures separately for `user0` through `user6`. For the generated log, the totals are 500 requests and 100 failures; each user has 14 failures except `user3` and `user5`, who have 15 each.

The first line, `#!/bin/bash`, is the shebang identifying the interpreter. The executable permission is enabled with `chmod +x`. The script is started with `./report.sh`, since the current working directory is not among the configured `PATH` entries. For a missing log file, the script writes its error to standard error using `>&2` and exits with status 1.

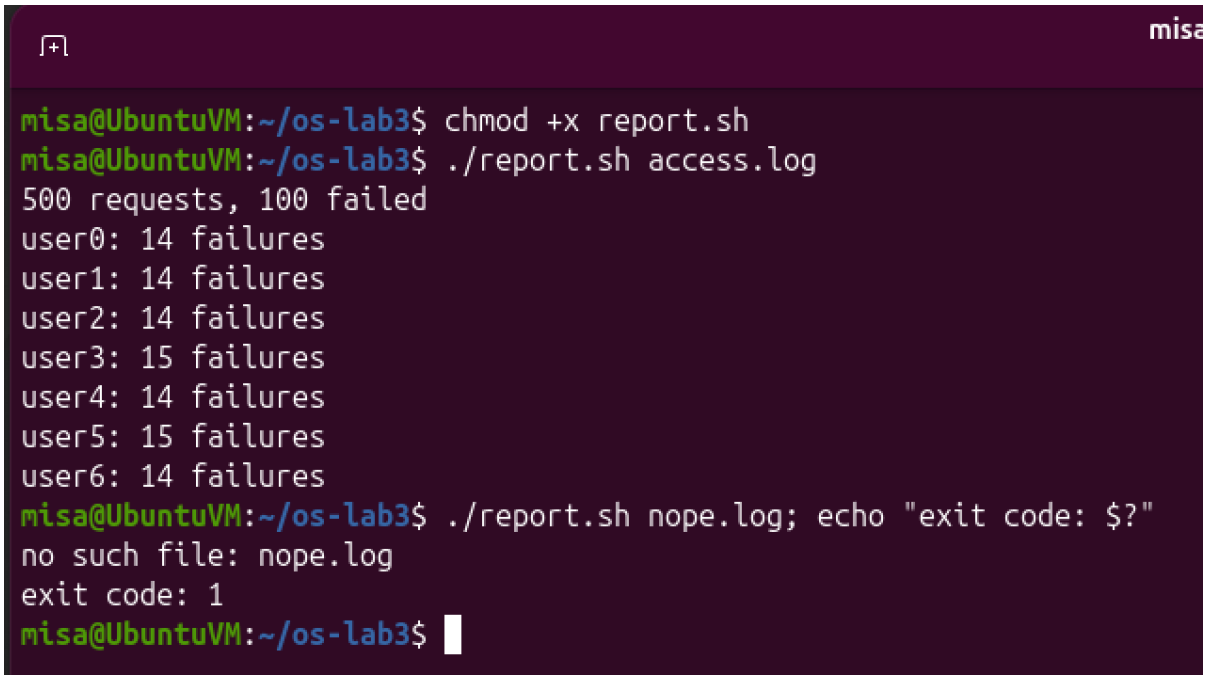


Figure 3.10 - Running report.sh with a valid and a missing input file

For the scheduling exercise, a user crontab entry runs the script once per minute using absolute paths. Its output is appended to `report.log`; `2>&1` also captures error messages. After two scheduled executions, the log is inspected and the added crontab entry is removed.

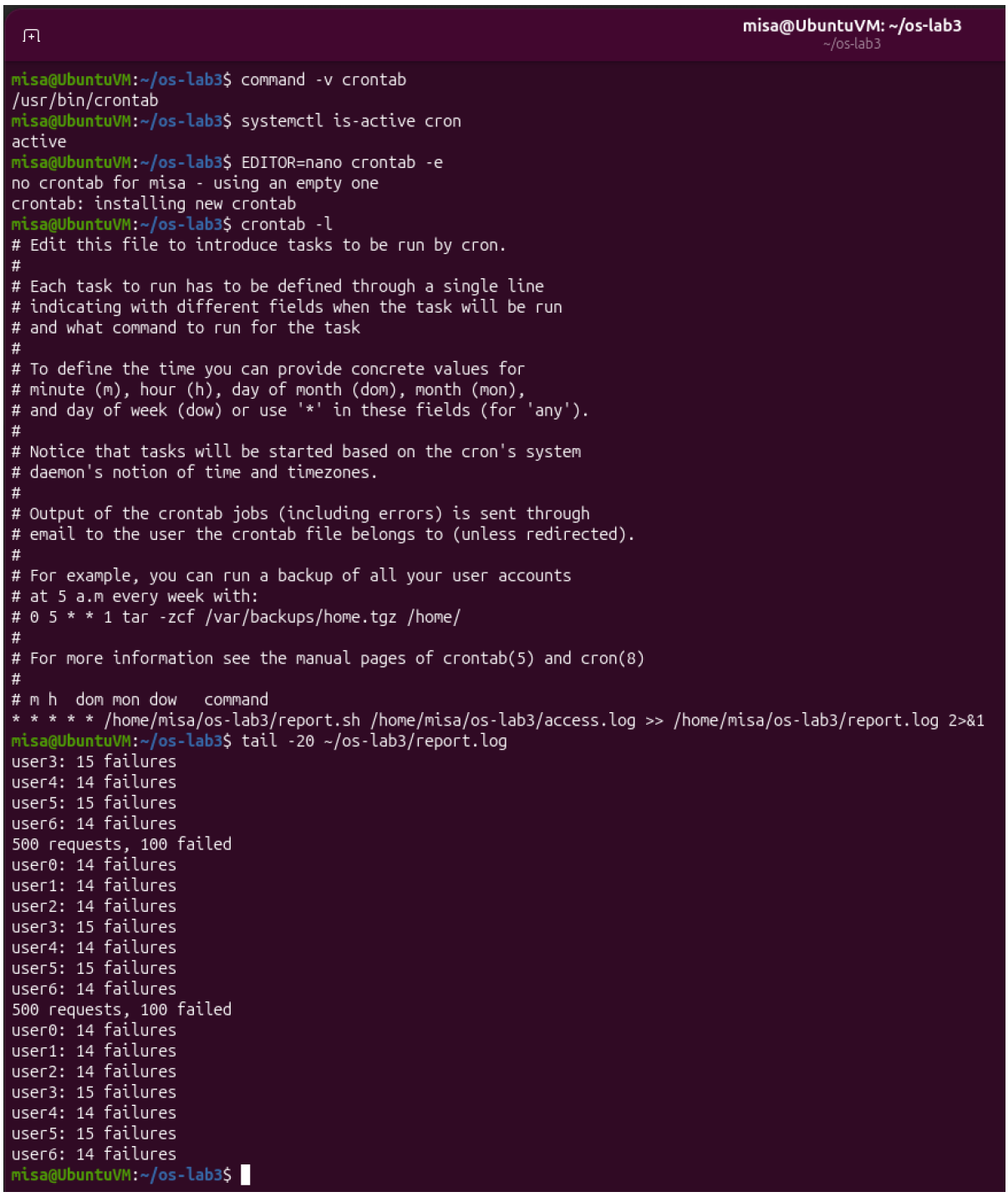


Figure 3.11 - Cron schedule and successive entries in report.log

```
misa@UbuntuVM:~/os-lab3$ EDITOR=nano crontab -e
crontab: installing new crontab
misa@UbuntuVM:~/os-lab3$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command

misa@UbuntuVM:~/os-lab3$
```

Figure 3.12 - Verifying removal of the temporary cron entry

4 CONCLUSIONS

The exercises showed how Bash transforms a command line before invoking a program and how its built-ins, command history, completion, and manual pages support efficient work in the terminal. Filename expansion, quoting, variable substitution, redirection, and conditional execution were observed directly in the shell.

Text filters and pipelines made it possible to extract useful statistics from a 500-line access log without manually examining each record. The resulting script combines those operations into a reusable report, including input validation and per-user failure counts. Scheduling that script with cron extends the same command-line techniques to recurring system tasks.